

C++ Training @ R&S

3. C++20 Concepts

Klaus Iglberger
27.4.-29.4.2026

The Placement of Constraints

```
using namespace std;
```

```
template< typename T >           // Need to restrict T
constexpr T                     // to integral types only
    sign( T a ) noexcept
{

    return ( is_signed_v<T> )
        ? ( T{0} < a ) - ( a < T{0} )
        : ( T{0} < a );
}
```

The Placement of Constraints

```
using namespace std;
```

```
#1 - Constraint template parameter
```

```
template< std::integral T >
```

```
constexpr T
```

```
    sign( T a ) noexcept
```

```
{
```

```
    return ( is_signed_v<T> )
```

```
        ? ( T{0} < a ) - ( a < T{0} )
```

```
        : ( T{0} < a );
```

```
}
```

The Placement of Constraints

```
using namespace std;
```

#2 - 'requires' clause after the template parameter list

```
template< typename T > requires std::integral<T>
```

```
constexpr T
```

```
    sign( T a ) noexcept
```

```
{
```

```
    return ( is_signed_v<T> )
```

```
        ? ( T{0} < a ) - ( a < T{0} )
```

```
        : ( T{0} < a );
```

```
}
```

The Placement of Constraints

```
using namespace std;
```

#3 - Abbreviated function template syntax

```
constexpr auto  
    sign( std::integral auto a ) noexcept  
{  
    using T = decltype(a);  
    return ( is_signed_v<T> )  
        ? ( T{0} < a ) - ( a < T{0} )  
        : ( T{0} < a );  
}
```

The Placement of Constraints

```
using namespace std;
```

#4 - Trailing 'requires' clause

```
constexpr auto  
    sign( auto a ) noexcept requires integral<decltype(a)>  
{  
    using T = decltype(a);  
    return ( is_signed_v<T> )  
        ? ( T{0} < a ) - ( a < T{0} )  
        : ( T{0} < a );  
}
```

Order of Constraint Evaluation

The constraints associated with a declaration are determined by normalizing a logical AND expression whose operands are in the following order:

- the constraint expression introduced for each **constrained template parameter**, in order of appearance;
- the constraint expression in the **requires** clause after the **template parameter list**;
- the constraint expression introduced for each parameter with constraint placeholder type in an **abbreviated function template declaration**;
- the constraint expression in the **trailing requires** clause.

Base Class vs. Concept

A base class expresses constraints and expectations for deriving types. A concept should do the same in the context of static polymorphism.

Bad concept: HasAdd queries for a property (syntax)

```
template< typename T >  
concept HasAdd =  
    requires( T t ) { t + t; };
```

Good concept: Addable describes constraints and expectations (semantics)

```
template< typename T >  
concept Addable =  
    requires( T t ) { t + t; };
```

Example: Constraining a Function Template

Task (B_Advanced_Templates/Concepts/Sign2): Constrain the `sign()` function to only integral data types by means of C++20 concepts.

Example: Constraining a Function Template

Task (B_Advanced_Templates/Concepts/Max3): Constrain the `max()` function by means of C++20 concepts to comparable types. In case both types are arithmetic, the comparison should be constraint according to the following rules:

- at least one of the two types is a floating point type;
- both types are either signed or unsigned.

Example: Constraining a Function Template

Task (B_Advanced_Templates/Concepts/SequenceContainer):

Step 1: Define the `SequenceContainer` concept to constrain the `addElement()` function such that for sequence containers the `push_back()` function is used and for associative containers the `insert()` function.

Step 2: Define the type trait `IsSequenceContainer`, including the according variable template.

Example: Constraining a Function Template

Task (B_Advanced_Templates/Concepts/AssociativeContainer):

Step 1: Define the `AssociativeContainer` concept to constrain the `addElement()` function such that for associative containers the `insert()` function is used and for sequence containers the `push_back()` function.

Step 2: Define the type trait `IsAssociativeContainer`, including the according variable template.

Example: Constraining Member Functions

Task (B_Advanced_Templates/Concepts/UniquePtr3): Constrain the move operations for different pointer types of the given `unique_ptr` implementation to convertible pointer types by means of C++20 concepts. Note that in this example you don't have to constrain the deleter!

Example: Constraining Constructors

Task (B_Advanced_Templates/Concepts/NarrowConversion2):
Constrain the `StrongType` constructor to prevent narrowing conversions by means of C++20 concepts.

Example: Assortment of Type Traits

Task (B_Advanced_Templates/Concepts/Constraints2):

Task 1: Implement the concept `IntCompatible` that requires the given type `T` to be convertible to `int` and to have a size greater than or equal to the size of `int`.

Task 2: Implement the concept `Arithmetic` that requires the given type `T` to be either an integral or floating point type, but does not accept `bool`.

Task 3: Implement the concept `IsNonConstReference` that requires the given type `T` to be a reference to a non-const type.

Task 4: Implement the concept `IntegralAddable` that requires the given type `T` to be addable. The result of the addition should yield an integral value.

Task 5: Implement the concept `NoNarrowingConversion` that prevents narrowing conversions between the given types `From` and `To`.

Task 6: Implement the concept `CompleteType` that requires the given type `T` to be a complete type.

Example: Constraining Destructors

Task (B_Advanced_Templates/Concepts/Optional_Trivial_1): Use C++20 concepts to guarantee that a specialization of `Optional` has the same destruction characteristics as its contained type. For trivial types, the destructor of `Optional` should be trivial, for non-trivial types it should be non-trivial and properly destroy the contained value.

std::optional Performance Analysis

Task (B_Advanced_Templates/Concepts/Optional_Trivial_2): Copy-and-paste the following code into godbolt.org. Compare the generated assembly code for the two `f()` functions. Which of the two `Optional` implementations is faster?

Programming Task

Task (B_Advanced_Templates/Concepts/IsPalindrome_Concepts):

Step 1: Implement the `is_palindrome()` algorithm in the following example. The algorithm should detect if the given range is the same when traversed forward and backward. The algorithm should return `true` only for true palindromes, and `false` for empty ranges and non-palindromes.

Step 2: Restrict the algorithm to bidirectional iterators by means of C++20 concepts.

Example: Constraining Destructors

```
template< typename T >
class Optional
{
public:
    constexpr Optional() = default;

    // For trivial types (e.g. 'int', 'double', 'float*', ...) the destructor of 'Optional'
    // should also be trivial.
    ~Optional() = default;           // This destructor is chosen for trivial types

    // For non-trivial types (e.g. 'std::string', ...) the following destructor should be
    // used, which makes 'Optional' non-trivial.
    ~Optional() requires( not std::is_trivially_destructible_v<T> )
    {
        if( used_ ) std::destroy_at( &value_ ); // This destructor is chosen for non-trivial
                                                // types since more constraint overloads are
                                                // preferred
    }

private:
    union { T value_; };
    bool used_{ false };
};

int main()
{
    static_assert(      std::is_trivially_destructible_v< Optional<int> > );
    static_assert( not std::is_trivially_destructible_v< Optional<std::string> > );
}
```

Concept Guidelines

Core Guideline T.10: Specify concepts for all template arguments

Core Guideline T.11: Whenever possible use standard concepts

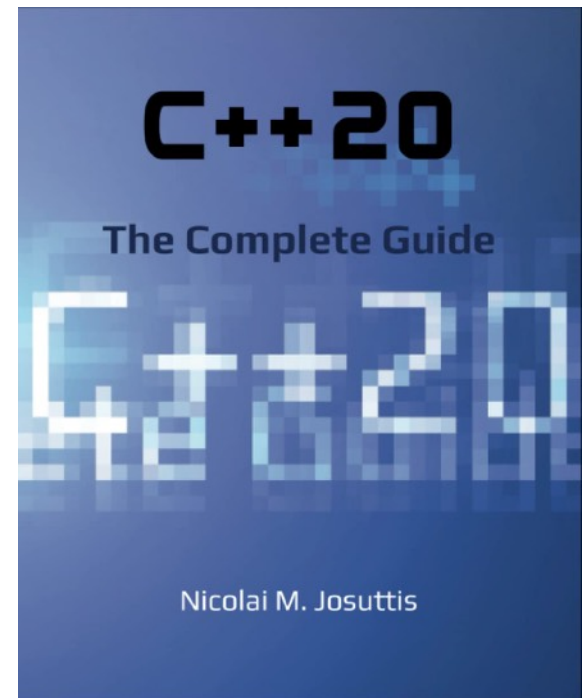
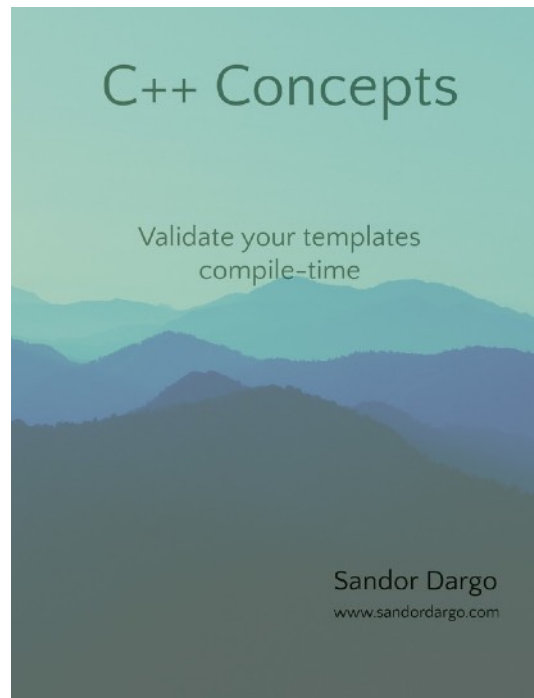
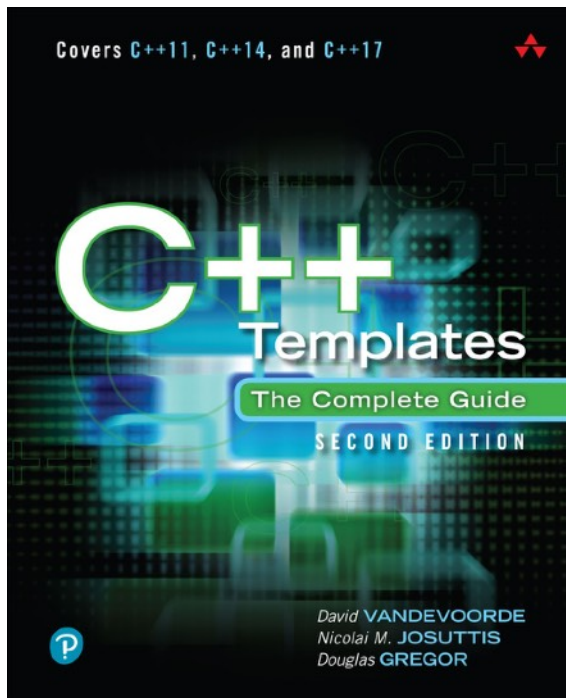
Core Guideline T.20: Avoid “concepts” without meaningful semantics

Core Guideline T.48: If your compiler does not support concepts, fake them with `enable_if`

Things to Remember

- Distinguish between rvalue references and forwarding references
- Use type traits for compile-time decisions
- Use SFINAE to express constraints
- Prefer C++20 concepts to SFINAE

Literature



References (Basic Templates)

- Andreas Fertig: Back to Basics: Templates (part 1 of 2). CppCon 2020 (<https://www.youtube.com/watch?v=VNJ4wiuxJM4>)
- Andreas Fertig: Back to Basics: Templates (part 2 of 2). CppCon 2020 (<https://www.youtube.com/watch?v=0dtjDTEE0hQ>)
- Walter Brown: Modern Template Metaprogramming: A Compendium, Part I. CppCon 2024 (<https://www.youtube.com/watch?v=Am2is2QCvxY>)
- Walter Brown: C++ Function Templates: How Do They Really Work?. CppCon 2018 (<https://www.youtube.com/watch?v=NIDEjY5ywqU>)
- Dan Saks: Back to Basics: Function and Class Templates. CppCon 2019 (https://www.youtube.com/watch?v=LMP_sxOaz6g)

References (Forwarding References)

- Nicolai Josuttis: Universal/Forwarding References - A Key to More Modern C++. C++ on Sea 2022 (https://www.youtube.com/watch?v=k1lz5u_IBZ4)

References (Type Traits)

- Jody Hagins: Template Metaprogramming: Type Traits (part 1 of 2). CppCon 2020 (<https://www.youtube.com/watch?v=tiAVWcjIF6o>)
- Jody Hagins: Template Metaprogramming: Type Traits (part 2 of 2). CppCon 2020 (<https://www.youtube.com/watch?v=dLZcocFOb5Q>)

References (SFINAE)

- Arthur O'Dwyer, “A Soupçon of SFINAE”. CppCon 2017 (<https://www.youtube.com/watch?v=ybaE9qlhHvw>)
- Steven Dewhurst, “Modern C++ Interfaces: Complexity, Emerging Simplicity, SFINAE, and Second Order Properties of Types”. CppCon 2017 (<https://www.youtube.com/watch?v=PFdWqa68LmA>)

References (Concepts)

- Nicolai Josuttis, “Back to Basics: Concepts”. CppCon 2024 (<https://www.youtube.com/watch?v=jzwqTi7n-rg>)
- Nina Ranns, “An Ode to Concepts”. CppCon 2024 (TBA)

email: klaus.iglberger@cpp-training.com

LinkedIn: [linkedin.com/in/klaus-iglberger](https://www.linkedin.com/in/klaus-iglberger)

Xing: [xing.com/profile/Klaus_Iglberger/cv](https://www.xing.com/profile/Klaus_Iglberger/cv)